

Sistema Escolar

Parte 4: Windows Forms, Observer y Reestructura del Proyecto

¿De dónde venimos y hacia dónde vamos?

En las tres partes anteriores construiste un sistema escolar con herencia, interfaces, Strategy, MVC y Repository. El sistema funciona correctamente, persiste datos y maneja errores de forma profesional. Pero tiene una limitación evidente: **la interfaz de usuario es la consola**.

La consola es un punto de partida válido para aprender, pero en el mundo real los usuarios esperan ventanas, botones, tablas y formularios. En esta parte vamos a dar ese salto: vamos a conectar todo lo que ya hiciste con una **interfaz gráfica en Windows Forms**.

Y lo más importante de esta parte es lo que vas a descubrir en el camino: **el Model no va a cambiar ni una línea**. Todo el esfuerzo que pusiste en separar responsabilidades va a dar sus frutos justo ahora.

Principio clave de esta parte

El Model no sabe nada de la interfaz. No sabe si es consola, Windows Forms o una app web.

Eso es posible porque en las partes anteriores separaste correctamente las capas. Ahora vas a ver el beneficio concreto: podés cambiar toda la interfaz de usuario sin tocar una sola línea de la lógica de negocio.

Paso 1 — Reestructurar el Proyecto

¿Por qué necesitamos dos proyectos?

Hasta ahora tenés un único proyecto de tipo **Console Application**. El problema es que Windows Forms es un tipo de aplicación diferente: no puede vivir dentro de un proyecto de consola.

Pero no podemos simplemente "convertir" el proyecto, porque eso significaría perder la app de consola. Y tampoco queremos copiar y pegar todas las clases: eso crearía duplicación de código, el peor enemigo del software mantenible.

La solución es la que ya usamos mentalmente cuando diseñamos el sistema: **separar la lógica de la presentación**. Solo que ahora lo vamos a hacer también a nivel de archivos en disco.

La caja de herramientas y el taller

Imaginá que tu lógica de negocio (Alumno, Materia, Repository, etc.) es una **caja de herramientas**. Esa caja no le pertenece a ningún taller en particular. Cualquier taller puede usarla.

Vas a crear dos "talleres": el taller de consola (que ya tenés) y el taller de Windows Forms (el nuevo). Ambos van a usar **la misma caja de herramientas**, sin duplicar nada.

En términos de C#: la caja de herramientas es un proyecto de tipo **Class Library** (.dll), y cada taller es un proyecto de tipo **Application** (.exe).

La estructura final de la solución

Al terminar el Paso 1, tu solución de Visual Studio va a tener esta estructura:

Proyecto	Tipo	Qué contiene
SistemaEscolar.Core	Class Library (.NET 5)	Toda la lógica: Persona, Alumno, Docente, Materia, interfaces, Strategy, Repository, DTOs, excepciones, Observer.
SistemaEscolar.UI	Windows Forms App (.NET)	Los formularios (Forms), el Controller adaptado a WinForms y el Program.cs de la interfaz gráfica.

Paso a paso en Visual Studio

Seguí estos pasos con cuidado. Cada uno está explicado para que entiendas qué estás haciendo, no solo qué botón presionar.

Parte A — Crear el proyecto Core (la caja de herramientas)

1. Abrió tu solución actual en Visual Studio 2019.
2. En el Explorador de Soluciones (panel derecho), hacé clic derecho sobre el **nombre de la Solución** (el ícono más arriba de todo, no sobre el proyecto).
3. Elegí **Agregar** → **Nuevo proyecto...**

4. En el buscador escribí "Class Library" y seleccioná "**Class Library (.NET Core)**" — asegurate de elegir la versión .NET Core, no la de .NET Framework.
5. Hacé clic en **Siguiente**.
6. Nombre del proyecto: `SistemaEscolar.Core`. Dejá la ubicación dentro de la misma carpeta de tu solución.
7. En la pantalla de framework, elegí **.NET 5.0**. Hacé clic en **Crear**.
8. Visual Studio va a crear el proyecto con un archivo `Class1.cs`. Borrarlo: clic derecho sobre él → Eliminar.

Parte B — Mover las clases existentes a Core

Ahora tenés que mover todos los archivos de lógica desde tu proyecto de consola hacia el nuevo proyecto Core.

¿Qué archivos mover? Todos los que no sean `Program.cs` ni nada de consola. Es decir:

`Persona.cs`, `Alumno.cs`, `Docente.cs`, `Materia.cs`

Toda la carpeta `Interfaces/`, `Strategy/`, `Repositorios/`, `DTOs/`, `Excepciones/`
`EscolarModel.cs` y `EstrategiaFactory.cs`

Cómo moverlos: En el Explorador de Soluciones, arrastrá cada archivo (o carpeta) desde el proyecto de consola hacia el proyecto `SistemaEscolar.Core`.

Alternativa si el drag-and-drop no funciona: hacé clic derecho sobre el archivo → Cortar → clic derecho en el proyecto Core → Pegar.

Importante: si tus clases tienen `namespace SistemaEscolar`, vas a necesitar que el Core también use ese mismo namespace, o actualizar los usings. Lo más simple es mantener el mismo namespace.

Parte C — Crear el proyecto de Windows Forms (el taller nuevo)

1. Nuevamente clic derecho sobre la **Solución** → **Agregar** → **Nuevo proyecto...**
2. En el buscador escribí "Windows Forms" y seleccioná "**Windows Forms App (.NET)**" — buscá el que dice ".NET" (sin "Framework").
3. Hacé clic en **Siguiente**.
4. Nombre del proyecto: `SistemaEscolar.UI`. Hacé clic en **Siguiente**.
5. Framework: **.NET 5.0**. Hacé clic en **Crear**.
6. Visual Studio crea el proyecto con un `Form1.cs` y un `Program.cs`. Renombrá `Form1.cs` a `FormPrincipal.cs` (clic derecho → Cambiar nombre).

Parte D — Conectar los dos proyectos (la referencia)

Para que el proyecto de Windows Forms pueda usar las clases de Core, hay que "conectarlos". Esto se llama **agregar una referencia de proyecto**.

1. En el Explorador de Soluciones, expandí el proyecto `SistemaEscolar.UI`.
2. Hacé clic derecho sobre "**Dependencias**" → **Agregar referencia de proyecto...**
3. Se abre una ventana con una lista de proyectos. Marcá la casilla de ``SistemaEscolar.Core``.

4. Hacer clic en **Aceptar**.

A partir de este momento, cualquier clase en `SistemaEscolar.UI` puede usar las clases de `SistemaEscolar.Core` con un simple `using SistemaEscolar.Core;` (o el namespace que hayas elegido).

¿Errores de compilación después de mover los archivos?

Es normal que aparezcan errores cuando movés los archivos. Los más comunes son:

"El tipo o nombre no existe": Revisá que todos los archivos fueron movidos al Core y que los namespaces son correctos.

"No se puede convertir...": Verificá que el proyecto UI tenga la referencia al Core (Parte D).

Tip: compilá primero solo el proyecto Core (clic derecho sobre él → Compilar) y verificá que no tiene errores antes de compilar la solución completa.

Ejercicio 1.A: Verificar la reestructura

1. Compilá el proyecto `SistemaEscolar.Core`. Debe compilar sin errores.
2. Verificá que el proyecto de consola original todavía funciona (si lo conservaste).
3. Verificá que el proyecto `SistemaEscolar.UI` puede ver las clases de Core (escribí "new Alumno" en `Form1.cs` y verificá que el autocompletado lo encuentra).
4. Anotá cuántos archivos terminaron en Core y cuántos en UI.

Ejercicio 1.B — Reflexión

- ¿Por qué `Alumno.cs`, `Materia.cs` y `EscolarModel.cs` van en Core y no en UI?
- ¿Qué pasaría si copiaras esas clases en lugar de moverlas? ¿Qué problema generaría?
- ¿Por qué crees que la referencia va de UI → Core y no al revés?

Paso 2 — Patrón de Diseño: Observer

¿Qué es el patrón Observer?

El patrón Observer resuelve un problema muy concreto: ¿cómo hace una parte del sistema para avisar a otras partes cuando algo cambia, **sin saber quiénes son esas otras partes**?

Pensá en las notificaciones de tu teléfono. Cuando una aplicación tiene algo nuevo para mostrarte, te envía una notificación. La app no sabe si tu pantalla está prendida o apagada, si estás mirando otra app, o si tenés el teléfono en silencio. Solo publica el evento. El sistema operativo se encarga de avisarle a quien corresponda.

La suscripción al diario

Imaginá un diario. El diario (el **sujeto**) publica noticias. Vos (el **observador**) te suscribís. Cada vez que sale una edición nueva, el diario te la manda automáticamente.

El diario no sabe quiénes son sus suscriptores ni qué hacen con el periódico. Solo sabe que tiene que enviarlo. Mañana podés cancelar la suscripción y el diario seguirá funcionando exactamente igual.

En nuestro sistema: `EscolarModel` es el diario. Los formularios de Windows Forms son los suscriptores.

¿Por qué no alcanza con los **eventos de C#** (los que ya usa WinForms)? Los eventos de C# son una implementación específica del patrón Observer, pero el Observer que vamos a construir es más explícito y didáctico. También nos permite que el Model (que vive en Core) notifique a los formularios (que viven en UI) **sin depender directamente de WinForms**.

Los tres actores del patrón Observer

Actor	Rol	En nuestro sistema
Sujeto (Subject)	El que tiene el estado y notifica cambios.	EscolarModel: tiene alumnos, materias y notifica cuando algo cambia.
Observador (Observer)	El que recibe las notificaciones y reacciona.	Los formularios de Windows Forms: actualizan la pantalla al recibir un aviso.
Evento (Event)	La descripción del cambio ocurrido.	Un enum CambioEscolar: AlumnoAgregado, MateriaAgregada, NotaCargada, etc.

Código: Implementar el Observer en el Core

Estos archivos van en el proyecto **SistemaEscolar.Core**. Son la infraestructura del patrón:

CambioEscolar.cs

```
// Describe qué tipo de cambio ocurrió en el sistema.
// Cada valor representa un evento de negocio.
public enum CambioEscolar
{
    AlumnoAgregado,
    DocenteAgregado,
```

```

    MateriaAgregada,
    AlumnoInscripto,
    NotaCargada,
    EstrategiaCambiada,
}

```

IObservadorEscolar.cs

```

// Contrato que deben cumplir todos los que quieran
// recibir notificaciones del sistema.
public interface IObservadorEscolar
{
    // cambio: qué ocurrió
    // dato: el objeto afectado (puede ser un Alumno, una Materia, etc.)
    void Actualizar(CambioEscolar cambio, object dato);
}

```

ISujetoEscolar.cs

```

// Contrato que debe cumplir cualquier clase que quiera
// tener observadores suscritos.
public interface ISujetoEscolar
{
    void Suscribirse(IObservadorEscolar observador);
    void Desuscribirse(IObservadorEscolar observador);
    void Notificar(CambioEscolar cambio, object dato);
}

```

Actualizar EscolarModel para implementar ISujetoEscolar

`EscolarModel` ahora implementa la interfaz `ISujetoEscolar`. Guardará una lista de observadores y los notificará automáticamente cada vez que algo cambie. Notá cómo el Model no sabe nada sobre formularios ni WinForms: solo conoce la interfaz `IObservadorEscolar`.

EscolarModel.cs — con Observer

```

public class EscolarModel : ISujetoEscolar
{
    private readonly IRepositorioAlumnos _repoAlumnos;
    private readonly IRepositorioMaterias _repoMaterias;
    // Lista de observadores suscritos
    private readonly List<IObservadorEscolar> _observadores = new();

    public EscolarModel(IRepositorioAlumnos repoAlumnos,
                       IRepositorioMaterias repoMaterias)
    {
        _repoAlumnos = repoAlumnos;
        _repoMaterias = repoMaterias;
    }

    // — Observer —————
    public void Suscribirse(IObservadorEscolar obs) => _observadores.Add(obs);
    public void Desuscribirse(IObservadorEscolar obs) => _observadores.Remove(obs);
    public void Notificar(CambioEscolar cambio, object dato)
    {
        // Notifica a cada observador registrado
        foreach (var obs in _observadores)
        {
            obs.Actualizar(cambio, dato);
        }
    }

    // — Operaciones del dominio (ahora notifican) —————
    public void AgregarAlumno(Alumno a)
    {

```

```

        _repoAlumnos.Agregar(a);
        Notificar(CambioEscolar.AlumnoAgregado, a);
    }

    public void AgregarMateria(Materia m)
    {
        _repoMaterias.Agregar(m);
        Notificar(CambioEscolar.MateriaAgregada, m);
    }

    public void InscribirAlumno(Alumno alumno, Materia materia)
    {
        materia.Inscribir(alumno);
        Notificar(CambioEscolar.AlumnoInscripto, alumno);
    }

    public void CargarNota(Alumno alumno, double nota)
    {
        alumno.AgregarNota(nota);
        Notificar(CambioEscolar.NotaCargada, alumno);
    }

    public List<Alumno> ObtenerAlumnos() => _repoAlumnos.ObtenerTodos();
    public List<Materia> ObtenerMaterias() => _repoMaterias.ObtenerTodos();

    public Alumno? BuscarAlumno(int legajo) =>
    _repoAlumnos.BuscarPorLegajo(legajo);
    public Materia? BuscarMateria(string codigo) =>
    _repoMaterias.BuscarPorCodigo(codigo);

    public void GuardarTodo()
    {
        _repoAlumnos.Guardar();
        _repoMaterias.Guardar();
    }
}

```

Ejercicio 2.A: Implementar Observer en Core

1. Creá los tres archivos: CambioEscolar.cs, IObservadorEscolar.cs e ISujetoEscolar.cs en el proyecto Core.
2. Actualizá EscolarModel para que implemente ISujetoEscolar.
3. Agregá llamadas a Notificar() en cada método que modifica datos.
4. Compilá el proyecto Core y verificá que no tiene errores.

Ejercicio 2.B — Reflexión

- ¿Por qué Notificar() recibe un object como segundo parámetro? ¿Qué ventaja tiene eso?
- ¿Qué pasaría si un observador lanza una excepción dentro de Actualizar()? ¿Cómo lo mejorarías?

Paso 3 — El Controller para Windows Forms

¿Por qué un Controller diferente?

En la Parte 2 creaste `EscolarController`, diseñado para leer desde la consola y mostrar en la consola. En WinForms la entrada y salida funcionan diferente: los datos vienen de `TextBox`, los errores se muestran con `MessageBox.Show()` y no hay un loop de menú.

El nuevo Controller, `EscolarControllerUI`, tiene una responsabilidad más simple: **recibir datos ya validados desde los formularios, ejecutar la operación en el Model y dejar que el Observer propague el resultado**. Es un intermediario más delgado.

¿Dónde va `EscolarControllerUI`?

Este archivo va en el proyecto **SistemaEscolar.UI**, no en Core. ¿Por qué? Porque podría necesitar referencias a WinForms (por ejemplo, para mostrar `MessageBox`). Si lo ponés en Core, obligarías a Core a depender de WinForms, lo cual rompería la separación.

Regla: **Core no sabe nada de UI. UI sabe todo de Core.**

`EscolarControllerUI.cs` (en `SistemaEscolar.UI`)

```
using SistemaEscolar.Core; // importa todas las clases del Core
using System.Windows.Forms;

public class EscolarControllerUI
{
    private readonly EscolarModel _model;

    public EscolarControllerUI(EscolarModel model)
    {
        _model = model;
    }

    // — Alumnos —————
    public bool AgregarAlumno(string nombre, string email, string legajoTexto)
    {
        try
        {
            int legajo = int.Parse(legajoTexto);
            _model.AgregarAlumno(new Alumno(nombre, email, legajo));
            _model.GuardarTodo();
            return true; // operación exitosa
        }
        catch (LegajoYaExisteException ex)
        {
            MessageBox.Show(ex.Message, "Legajo duplicado",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
        catch (FormatException)
        {
            MessageBox.Show("El legajo debe ser un número entero.",
                "Dato inválido", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }

    public bool EliminarAlumno(Alumno alumno)
    {
        var confirmacion = MessageBox.Show(
            $"¿Eliminar al alumno {alumno.Nombre}?",
            "Confirmar eliminación",
```



```

        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

        if (confirmacion != DialogResult.Yes) return false;

        // Para eliminar necesitamos agregar el método al repositorio (ver Ejercicio
3.B)
        _model.EliminarAlumno(alumno);
        _model.GuardarTodo();
        return true;
    }

    // — Materias —————
    public bool AgregarMateria(string codigo, string nombre, string cupoTexto)
    {
        try
        {
            int cupo = int.Parse(cupoTexto);
            _model.AgregarMateria(new Materia(codigo, nombre, cupo));
            _model.GuardarTodo();
            return true;
        }
        catch (FormatException)
        {
            MessageBox.Show("El cupo debe ser un número entero.",
                "Dato inválido", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }

    // — Inscripciones y notas —————
    public bool InscribirAlumno(Alumno alumno, Materia materia)
    {
        try
        {
            _model.InscribirAlumno(alumno, materia);
            _model.GuardarTodo();
            return true;
        }
        catch (SinCupoException ex)
        {
            MessageBox.Show(ex.Message, "Sin cupo",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message, "Operación inválida",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
    }

    public void CargarNota(Alumno alumno, double nota)
    {
        _model.CargarNota(alumno, nota);
        _model.GuardarTodo();
    }
}

```

Ejercicio 3.A: Crear EscolarControllerUI

1. Creá `EscolarControllerUI.cs` en el proyecto `SistemaEscolar.UI`.
2. Verificá que los usings son correctos (necesitás using para el Core y para WinForms).
3. Compilá el proyecto UI y verificá que no tiene errores.

Ejercicio 3.B — DESAFÍO: Eliminar alumnos

Para poder eliminar un alumno, el repositorio necesita un método Eliminar(Alumno a).

Agregá Eliminar(T entidad) a la interfaz IRepository<T>.

Implementalo en RepositorioAlumnosMemoria y RepositorioAlumnosJson.

Agregá EliminarAlumno(Alumno a) en EscolarModel que llama al repo y notifica con un nuevo CambioEscolar.AlumnoEliminado.

Paso 4 — El Formulario Principal

Entendiendo Windows Forms antes de empezar

Antes de escribir código, es importante entender cómo funciona WinForms para no perderse en los detalles.

Los tres pilares de WinForms

- 1. El Diseñador (Designer):** Una herramienta visual donde arrastrás y soltás controles (botones, tablas, campos de texto) para construir la ventana. Visual Studio genera el código equivalente automáticamente.
- 2. Los Controles:** Cada elemento visual es un objeto: `Button`, `TextBox`, `DataGridView`, `Label`, `TabControl`. Tienen propiedades (texto, color, tamaño) y métodos.
- 3. Los Eventos:** Cuando el usuario hace clic en un botón, mueve el mouse o escribe en un campo, WinForms genera un evento. Vos escribís el código que se ejecuta cuando ese evento ocurre (por ejemplo, `btnGuardar_Click`).

Diseñar el formulario principal en el Designer

El `FormPrincipal` es la ventana central de la aplicación. Contiene un `TabControl` con dos pestañas: una para Alumnos y otra para Materias. En cada pestaña hay una tabla (`DataGridView`) que muestra los datos y botones para las operaciones CRUD.

Construir FormPrincipal en el Designer

- Hacé doble clic sobre `FormPrincipal.cs` en el Explorador de Soluciones. Se abre el diseñador.
- En el panel **Propiedades** (esquina inferior derecha), cambiá la propiedad `Text` del formulario a "Sistema Escolar" y `Size` a 900, 600.
- Desde el **Cuadro de herramientas** (panel izquierdo), arrastrá un **TabControl** al formulario. Estíralo para que ocupe casi todo el espacio.
- El `TabControl` trae dos tabs. Hacé clic derecho sobre el `TabControl` → **Agregar pestaña** si necesitás más. Seleccioná la primera pestaña y en **Propiedades** cambiá su `Text` a "Alumnos". La segunda a "Materias".
- En la pestaña Alumnos, arrastrá un **DataGridView**. Estíralo para que ocupe la mayor parte de la pestaña. En **Propiedades**, nombralo `dgvAlumnos`. Deshabilitá la edición directa: propiedad `ReadOnly = True`.
- Debajo del `DataGridView`, arrastrá tres **Button**. Nombralos `btnAgregarAlumno`, `btnEditarAlumno` y `btnEliminarAlumno`. Cambiá su propiedad `Text` a "Agregar", "Editar" y "Eliminar".
- Repetí los pasos 5 y 6 en la pestaña Materias, con `dgvMaterias`, `btnAgregarMateria`, `btnEditarMateria` y `btnEliminarMateria`.
- Agregá también un botón `btnInscribir` con texto "Inscribir Alumno" en la pestaña Materias.

El código de FormPrincipal

Una vez diseñado el formulario, abrí el archivo de código (`FormPrincipal.cs`, haciendo clic derecho → "Ver código"). Vas a ver que `InitializeComponent()` ya está generado por el Designer. Agregá el resto:

FormPrincipal.cs

```
using SistemaEscolar.Core;
using System;
using System.Windows.Forms;

// FormPrincipal implementa IObservadorEscolar:
// cuando el Model cambia, este formulario se actualiza solo.
public partial class FormPrincipal : Form, IObservadorEscolar
{
    private readonly EscolarModel _model;
    private readonly EscolarControllerUI _controller;

    public FormPrincipal(EscolarModel model, EscolarControllerUI controller)
    {
        InitializeComponent(); // código generado por el Designer
        _model = model;
        _controller = controller;

        // Nos suscribimos al Model para recibir notificaciones
        _model.Suscribir(this);

        // Cargamos los datos al abrir
        CargarAlumnos();
        CargarMaterias();
    }

    // — Observer: este método se llama automáticamente cuando algo cambia —
    public void Actualizar(CambioEscolar cambio, object dato)
    {
        // Usamos Invoke para ejecutar en el hilo de la UI (requerido en WinForms)
        if (InvokeRequired)
        {
            Invoke(new Action(() => Actualizar(cambio, dato)));
            return;
        }

        switch (cambio)
        {
            case CambioEscolar.AlumnoAgregado:
            case CambioEscolar.AlumnoEliminado:
                CargarAlumnos();
                break;
            case CambioEscolar.MateriaAgregada:
                CargarMaterias();
                break;
            case CambioEscolar.AlumnoInscripto:
                CargarMaterias(); // actualiza el contador de inscriptos
                break;
        }
    }

    // — Carga de datos en las tablas —
    private void CargarAlumnos()
    {
        dgvAlumnos.DataSource = null;
        dgvAlumnos.DataSource = _model.ObtenerAlumnos();
        // Ajustamos las columnas para que se lean bien
        if (dgvAlumnos.Columns.Count > 0)
        {
            dgvAlumnos.AutoResizeColumns();
        }
    }

    private void CargarMaterias()
    {

```

```

    {
        dgvMaterias.DataSource = null;
        dgvMaterias.DataSource = _model.ObtenerMaterias();
        if (dgvMaterias.Columns.Count > 0)
            dgvMaterias.AutoResizeColumns();
    }

    // — Eventos de botones — Alumnos —————
    private void btnAgregarAlumno_Click(object sender, EventArgs e)
    {
        // Abre el formulario de alta. Al cerrar, el Observer ya actualizó la tabla.
        using var form = new FormAlumno(_controller);
        form.ShowDialog();
    }

    private void btnEliminarAlumno_Click(object sender, EventArgs e)
    {
        if (dgvAlumnos.CurrentRow?.DataBoundItem is Alumno alumno)
            _controller.EliminarAlumno(alumno);
        else
            MessageBox.Show("Seleccioná un alumno de la tabla.",
                            "Sin selección", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }

    // — Eventos de botones — Materias —————
    private void btnAgregarMateria_Click(object sender, EventArgs e)
    {
        using var form = new FormMateria(_controller);
        form.ShowDialog();
    }

    private void btnInscribir_Click(object sender, EventArgs e)
    {
        if (dgvMaterias.CurrentRow?.DataBoundItem is Materia materia)
        {
            using var form = new FormInscripcion(_controller, _model, materia);
            form.ShowDialog();
        }
        else
            MessageBox.Show("Seleccioná una materia de la tabla.",
                            "Sin selección", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }

    // Liberar la suscripción al cerrar el formulario
    protected override void OnFormClosed(FormClosedEventArgs e)
    {
        _model.GuardarTodo();
        _model.Desuscribir(this);
        base.OnFormClosed(e);
    }
}

```

¿Qué es InvokeRequired?

En WinForms, **solo el hilo principal puede actualizar la interfaz gráfica**. Si el Observer notificara desde otro hilo (por ejemplo, en una app más compleja), intentar actualizar una tabla desde ese hilo causaría un error.

El patrón `if (InvokeRequired) { Invoke(...); return; }` es la forma estándar de WinForms para asegurarse de que el código de actualización siempre corra en el hilo correcto. Por ahora puede parecer repetitivo, pero es una buena práctica incluirlo desde el inicio.

Paso 5 — Formularios de CRUD

FormAlumno: agregar alumnos

Este formulario es una ventana pequeña que el usuario abre cuando quiere agregar un alumno. Tiene campos de texto para los datos y botones de Guardar y Cancelar.

Diseñar FormAlumno en el Designer

1. En el proyecto UI, clic derecho → Agregar → Formulario de Windows Forms. Nombre: `FormAlumno.cs`.
2. Cambiá el `Text` del formulario a "Agregar Alumno" y el `Size` a 380, 260. Establecé `FormBorderStyle = FixedDialog` y `MaximizeBox = False`.
3. Agregá tres `Label` con los textos "Nombre:", "Email:" y "Legajo:"
4. Junto a cada `Label`, agregá un `TextBox`: nombralos `txtNombre`, `txtEmail` y `txtLegajo`.
5. Agregá dos botones: `btnGuardar` (Text: "Guardar") y `btnCancelar` (Text: "Cancelar").
6. En el botón Cancelar, establecé la propiedad `DialogResult = Cancel`. En Guardar, `DialogResult = None` (el código va a cerrarlo manualmente).

FormAlumno.cs

```
using SistemaEscolar.Core;
using System;
using System.Windows.Forms;

public partial class FormAlumno : Form
{
    private readonly EscolarControllerUI _controller;

    public FormAlumno(EscolarControllerUI controller)
    {
        InitializeComponent();
        _controller = controller;
    }

    private void btnGuardar_Click(object sender, EventArgs e)
    {
        // Validación básica de campos vacíos antes de llamar al controller
        if (string.IsNullOrEmpty(txtNombre.Text) ||
            string.IsNullOrEmpty(txtEmail.Text) ||
            string.IsNullOrEmpty(txtLegajo.Text))
        {
            MessageBox.Show("Todos los campos son obligatorios.",
                "Campos incompletos",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        // El controller hace la validación de dominio (legajo único, etc.)
        // y muestra MessageBox si hay un error.
        // Si retorna true, la operación fue exitosa: cerramos el formulario.
        bool exito = _controller.AgregarAlumno(
            txtNombre.Text.Trim(),
            txtEmail.Text.Trim(),
            txtLegajo.Text.Trim()
        );

        if (exito)
            Close(); // FormPrincipal ya se actualizó via Observer
    }

    private void btnCancelar_Click(object sender, EventArgs e)
    {

```

```

        Close();
    }
}

```

FormMateria: agregar materias

Mismo patrón que FormAlumno, pero con los campos de Materia: Código, Nombre y Cupo.

FormMateria.cs

```

using SistemaEscolar.Core;
using System.Windows.Forms;

public partial class FormMateria : Form
{
    private readonly EscolarControllerUI _controller;

    public FormMateria(EscolarControllerUI controller)
    {
        InitializeComponent();
        _controller = controller;
    }

    private void btnGuardar_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(txtCodigo.Text) ||
            string.IsNullOrEmpty(txtNombre.Text) ||
            string.IsNullOrEmpty(txtCupo.Text))
        {
            MessageBox.Show("Todos los campos son obligatorios.",
                "Campos incompletos",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        bool exito = _controller.AgregarMateria(
            txtCodigo.Text.Trim(),
            txtNombre.Text.Trim(),
            txtCupo.Text.Trim()
        );

        if (exito) Close();
    }

    private void btnCancelar_Click(object sender, EventArgs e) => Close();
}

```

FormInscripcion: inscribir un alumno en una materia

Este formulario recibe la materia ya seleccionada desde el formulario principal. Muestra un [ComboBox](#) con la lista de alumnos disponibles para elegir cuál inscribir.

Diseñar FormInscripcion en el Designer

1. Creá `FormInscripcion.cs`. `Size: 380, 200, FormBorderStyle = FixedDialog`.
2. Agregá un **Label** con texto "Seleccionar alumno:".
3. Agregá un **ComboBox**, nombralo `cboAlumnos`. `DropDownStyle = DropDownList` (solo se puede elegir, no escribir).
4. Agregá botones `btnInscribir` (Text: "Inscribir") y `btnCancelar` (Text: "Cancelar").

FormInscripcion.cs

```
using SistemaEscolar.Core;
using System.Linq;
using System.Windows.Forms;

public partial class FormInscripcion : Form
{
    private readonly EscolarControllerUI _controller;
    private readonly EscolarModel _model;
    private readonly Materia _materia;

    public FormInscripcion(EscolarControllerUI controller,
                           EscolarModel model, Materia materia)
    {
        InitializeComponent();
        _controller = controller;
        _model = model;
        _materia = materia;

        // Cargamos la lista de alumnos en el ComboBox.
        // Mostramos el nombre pero el objeto completo queda guardado.
        cboAlumnos.DataSource = _model.ObtenerAlumnos();
        cboAlumnos.DisplayMember = "Nombre"; // qué texto mostrar
        cboAlumnos.ValueMember = "Legajo"; // qué valor usar internamente

        Text = $"Inscribir en: {materia.Nombre}";
    }

    private void btnInscribir_Click(object sender, EventArgs e)
    {
        if (cboAlumnos.SelectedItem is Alumno alumno)
        {
            bool exito = _controller.InscribirAlumno(alumno, _materia);
            if (exito) Close();
        }
    }

    private void btnCancelar_Click(object sender, EventArgs e) => Close();
}
```


Paso 6 — Conectar Todo en Program.cs

El punto de entrada de la aplicación de Windows Forms

En una aplicación de WinForms, `Program.cs` cumple el mismo rol que en la consola: es el punto de entrada. Acá se configura todo antes de mostrar la primera ventana.

La diferencia clave con la consola es que en lugar de llamar a `controller.Ejecutar()` (que tenía un loop), llamamos a `Application.Run(form)`, que es el loop de eventos de WinForms: espera interacciones del usuario indefinidamente hasta que se cierra la ventana.

Program.cs (en SistemaEscolar.UI)

```
using SistemaEscolar.Core;
using System;
using System.Windows.Forms;

static class Program
{
    [STAThread] // requerido por WinForms para el modelo de threading
    static void Main()
    {
        // Configuración estándar de WinForms
        Application.SetHighDpiMode(HighDpiMode.SystemAware);
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        // — 1. Crear los repositorios —————
        // Usamos JSON para que los datos persistan entre sesiones.
        // Si querés empezar con datos en memoria para probar,
        // reemplazá por RepositorioAlumnosMemoria() y RepositorioMateriasMemoria().
        var repoAlumnos = new RepositorioAlumnosJson("alumnos.json");
        var repoMaterias = new RepositorioMateriasJson("materias.json", repoAlumnos);

        // — 2. Crear el Model —————
        var model = new EscolarModel(repoAlumnos, repoMaterias);

        // — 3. Crear el Controller —————
        var controller = new EscolarControllerUI(model);

        // — 4. Crear y mostrar el formulario principal —————
        // El formulario se suscribe al Model en su constructor.
        var formPrincipal = new FormPrincipal(model, controller);

        // Application.Run inicia el loop de eventos de WinForms.
        // La app corre hasta que el usuario cierra la ventana.
        Application.Run(formPrincipal);
    }
}
```

¿Qué es Application.Run?

Es el corazón de cualquier aplicación Windows Forms. Inicia un **bucle de eventos** que está constantemente escuchando: clicks del mouse, teclas presionadas, mensajes del sistema operativo.

Cuando el usuario cierra la ventana principal, `Application.Run` termina y el programa finaliza. Por eso guardamos los datos en el evento `OnFormClosed` del formulario principal.

Conectar los eventos del Designer al código

Para que los botones funcionen, hay que conectar cada botón a su método de código. En el Designer esto se hace así:

Conectar un evento Click en el Designer

1. En el Designer, hacé **doble clic** sobre el botón `btnAgregarAlumno`.
2. Visual Studio crea automáticamente el método `btnAgregarAlumno_Click` en el código y te lleva hasta él.
3. Dentro del método, escribí el código correspondiente (el que ya está en `FormPrincipal.cs` arriba).

Alternativa: en el panel **Propiedades**, hacé clic en el ícono del rayo (⚡ Eventos). Encontrás el evento `Click` y podés escribir el nombre del método manualmente o elegir uno existente.

Importante: cada botón de cada formulario necesita que conectes su evento Click de esta forma.

Ejercicio 6.A: Armar la aplicación completa

1. Creá y diseñá `FormPrincipal`, `FormAlumno`, `FormMateria` y `FormInscripcion` en el Designer.
2. Implementá el código de cada formulario.
3. Configurá `Program.cs` con los repositorios, model, controller y formulario principal.
4. Ejecutá la aplicación (F5) y verificá que podés agregar y ver alumnos y materias.
5. Cerrá y volvé a abrir la aplicación. ¿Los datos persisten?

Ejercicio 6.B — DESAFÍO: Formulario de notas

Creá un `FormNota` que permita seleccionar un alumno desde un `ComboBox` e ingresar una nota con un `NumericUpDown` (rango 1-10, con decimales).

Mostrá el promedio actual del alumno en un `Label` que se actualice al cargar una nueva nota.

Tip: suscribí el `FormNota` al `Model` y actualizá el `Label` en el método `Actualizar()` cuando recibas `CambioEscolar.NotaCargada`.

Paso 7 — El Observer en Acción: Verificar que Todo Funciona

Trazando el flujo completo de una operación

Para entender bien cómo se conectan todas las piezas, vamos a trazar el camino completo de lo que pasa cuando el usuario hace clic en "Agregar" y carga un alumno nuevo:

Pas o	Qué ocurre	Quién lo hace
1	El usuario rellena los campos y hace clic en "Guardar".	FormAlumno (View)
2	El formulario llama a <code>_controller.AgregarAlumno(...)</code> .	FormAlumno (View)
3	El Controller valida y llama a <code>_model.AgregarAlumno(new Alumno(...))</code> .	EscolarControllerUI
4	EscolarModel agrega el alumno al repositorio.	EscolarModel (Model)
5	EscolarModel llama a <code>Notificar(AlumnoAgregado, alumno)</code> .	EscolarModel (Model)
6	<code>Notificar()</code> recorre la lista de observadores y llama a <code>Actualizar()</code> en cada uno.	EscolarModel (Observer)
7	<code>FormPrincipal.Actualizar()</code> recibe el evento y llama a <code>CargarAlumnos()</code> .	FormPrincipal (Observer)
8	<code>CargarAlumnos()</code> actualiza el DataGridView. El usuario ve el nuevo alumno en la tabla.	FormPrincipal (View)
9	FormAlumno se cierra automáticamente (el Controller retornó true).	FormAlumno (View)

¿Qué pasaría sin el Observer?

Sin Observer, después de agregar el alumno habría que llamar manualmente a `formPrincipal.CargarAlumnos()` desde el controller, o desde FormAlumno. Eso significa que el controller o el formulario hijo necesitarían tener una referencia al formulario principal.

Eso genera **acoplamiento**: si mañana agregás una segunda ventana que también muestra alumnos, tendrías que actualizar el controller para que llame a ambas. Con Observer, simplemente suscribís la segunda ventana y el Model la notificará automáticamente.

Consejos para depurar la aplicación

Si algo no funciona como esperás, estos son los problemas más comunes:

- **La tabla no se actualiza:** Verificá que el formulario está suscripto al Model (`_model.Suscribir(this)` en el constructor) y que los casos del `switch` en `Actualizar()` coinciden con los eventos que lanza el Model.
- **Error "cross-thread operation":** Asegurate de incluir el bloque `if (InvokeRequired)` { `Invoke(...); return;` } al principio de `Actualizar()`.

- **Los datos no persisten:** Verificá que `GuardarTodo()` se llama antes de cerrar la aplicación (en `OnFormClosed`) y que los archivos JSON se están creando en el directorio correcto.
- **El ComboBox no muestra nada:** Verificá que `DisplayMember` coincide exactamente con el nombre de la propiedad (incluyendo mayúsculas): `"Nombre"`, no `"nombre"`.
- **Namespace no encontrado:** Si ves errores de "tipo no existe", revisá que el proyecto UI tiene la referencia al Core y que los `using` están correctos.

Resumen: Estructura Final del Proyecto Completo

Al terminar esta guía, la solución tiene **dos proyectos** con esta distribución de archivos:

SistemaEscolar.Core (Class Library)

Archivo / Carpeta	Descripción
Persona.cs / Alumno.cs / Docente.cs / Materia.cs	Clases del dominio (sin cambios respecto a Parte 3)
Interfaces/ (ICalificable, IInscribible, IReportable, IRepositorio...)	Contratos de comportamiento y repositorios
Strategy/ (IEstrategiaPromedio, PromedioSimple, etc.)	Implementaciones del patrón Strategy
Repositorios/ (Memoria y JSON)	Acceso a datos con el patrón Repository
DTOs/ (AlumnoDto, MateriaDto)	Clases de transferencia para JSON
Excepciones/ (LegajoYaExiste, SinCupo, EntidadNoEncontrada)	Errores del dominio
CambioEscolar.cs	Enum con los tipos de eventos del Observer
IObservadorEscolar.cs	Interfaz que implementan los formularios
ISujetoEscolar.cs	Interfaz que implementa EscolarModel
EscolarModel.cs	Model central: coordina repositorios y notifica cambios
EstrategiaFactory.cs	Factory para reconstruir estrategias desde JSON

SistemaEscolar.UI (Windows Forms App)

Archivo	Descripción
Program.cs	Ensambla repositorios, model, controller y lanza la app WinForms
EscolarControllerUI.cs	Controller adaptado a WinForms: valida datos y muestra MessageBox
FormPrincipal.cs / .Designer.cs	Ventana principal con TabControl, DataGridViews y botones. Implementa IObservadorEscolar.
FormAlumno.cs / .Designer.cs	Formulario de alta de alumnos
FormMateria.cs / .Designer.cs	Formulario de alta de materias
FormInscripcion.cs / .Designer.cs	Formulario para inscribir un alumno en una materia